旧方案有平均收益，也存在严重反例

本次结论修正了此前仅用单一固定种子、200组相似负载得到的判断。旧算法没有表现为普遍稳健的高分方案，不能根据平均收益宣称它适用于所有隐藏样例。

主要证据	实际结果
独立输入 / 输入操作	1,309 个 / 21,497,848 条
记录的新随机种子	1,012 个，包含两批独立种子的验证数据
执行方式	真实C++调度器，10,460 次回放；不再仅验证Python译写模型
合法性	0个意外失败；1个预期不可行输入在8个策略上失败，已单列
最大规模	3个输入各含500万条操作，K=8、P=64；含B=1和4GiB容量边界

994个题面范围内可行输入中的比较

策略	几何平均相对代价	最坏相对代价
旧版 / 周期预取	0.678212	1166.50倍
V2 / 激进双目标	0.634343	1166.50倍
Prefix / 保守已知前缀	0.998469	1.00倍

相对代价 = 算法Score / 安全朴素基线Score，越低越好。31个基线Score=0的输入不参与比值计算，仍参与正确性检查。表中比值实际基于963个正分输入。

选择建议

追求本组合成分布的平均收益，激进双目标版更好；重视避免已发现的严重退化，保守Prefix版更适合做提交底座，但收益明显较小。当前没有证据支持“一个已证明适用于全部赛题样例的最优算法”。

测试不靠挑种子，而靠分布与边界覆盖

固定随机种子本身是可复现性的要求。问题在于只测试一种负载分布、一个随机序列，以及缺少独立验证和最坏情况检查。增加种子数量也不能替代构造反例和参数边界。

数据组	数量	用途
开发分布	160	20类负载，每类8个新种子；报告全部结果
独立验证A	320	20类负载，每类16个新种子
独立验证B	320	另一批独立系统熵，每类16个新种子
极端参数矩阵	128	K、P、M、W、R、penalty、B各取两个端点，完整2^7组合
边界输入	6	恰好M、M-1、当前分区增长、空分区、无Probe、末尾Switch
语义变换	60	20组原输入、同分区不同key、算子/分区ID旋转
小规模精确模型	307	3^5条Probe序列完整枚举243例，加64个随机小输入
预期不可行诊断	1	最终需加载的原子分区大于M
确定性反例与回归	4	针对BUILD决策、付费保护、相邻双Probe
最大规模压力	3	每例500万条操作

本轮证据与探索数据分开

先前探索批次暴露8317倍反例后，增加保守Prefix策略；随后冻结策略源文件，再用新的系统随机熵重新生成本报告主验证批次。主批次没有按结果修改算法或筛选种子。探索批次原文件也保留，不能把它们混称为未见验证数据。

entropy.json记录系统熵和源文件SHA-256；manifest.json记录每例种子、参数、输入摘要。final_audit.json已逐个核对1309个输入及冻结源文件的摘要。

覆盖20类负载，不只测试固定热点

负载	旧版几何相对代价	激进版几何相对代价
接近均匀但保留大小倾斜	0.742	0.707
Zipf式大小与访问倾斜	0.710	0.712
小分区热点	1.076	0.992
大分区热点	0.058	0.056
热点按阶段迁移	0.587	0.569
短突发与重复访问	0.721	0.705
工作集循环与容量争用	0.465	0.432
全分区顺序扫描	1.027	1.039
稳定热点夹杂冷扫描	0.917	0.884
相邻两热点能够共同驻留	0.579	0.059
两个热点总大小超过M	0.994	0.994
多算子轮换争用内存	0.671	0.660
部分算子Probe与其他算子Build交叠	0.726	0.729
Build主导、Probe较少	0.683	0.658
一个分区接近或恰好M	0.997	0.996
大量空分区及空分区Probe	0.751	0.741
热点后接一次性扫描尾部	0.899	0.870
读慢写快与读快写慢	0.727	0.700
极小热点夹杂巨大冷分区	1.000	1.000
分区大小位于容量临界点±1字节	0.567	0.565

每类40例，包含开发和两批验证种子。几何平均仅描述本测试混合分布。不同家族的改善与退化均保留，不能只选择收益较大的热点类型。

平均数之外，同时看输赢次数与尾部风险

策略	胜 / 平 / 负	几何均值	P95比值	最坏比值
安全朴素基线	0 / 963 / 0	1.0000	1.000	1.00
旧版 / 无预取	600 / 139 / 224	0.6826	1.304	1166.50
旧版 / 周期预取	626 / 119 / 218	0.6782	1.278	1166.50
惰性衰减 / 等价	626 / 119 / 218	0.6782	1.278	1166.50
V2 / 空间搭车预取	636 / 110 / 217	0.6783	1.272	1166.50
V2 / 下一目标保护	649 / 110 / 204	0.6373	1.239	1166.50
V2 / 激进双目标	653 / 110 / 200	0.6343	1.232	1166.50
Prefix / 保守已知前缀	513 / 450 / 0	0.9985	1.000	1.00

独立验证的结果与不确定性

策略	几何相对代价	95%重采样区间
旧版 / 周期预取	0.67571	0.62318 – 0.72996
V2 / 激进双目标	0.62715	0.57605 – 0.68117
Prefix / 保守已知前缀	0.99894	0.99869 – 0.99918

A、B两批验证共640例，其中618例的基线分为正。区间由2000次成对重采样计算，只说明这些合成家族与种子的均值稳定程度，不是官方隐藏数据的置信保证。

计分口径

每例严格计算 Score = 写字节/W + 读字节/R + penalty×Miss。逐例BASE/VAL的平均值也保存于JSON，但这里的BASE是修复兼容问题、固定平局规则的本地基线。题面官方BASE未知，不能将该平均值乘50宣称已获比赛分数。

不能把所有样例的原始微秒直接相加后只报一个百分比：读写带宽差四个数量级，少数大代价样例会主导总和。报告同时提供几何平均、逐例输赢、P95、最坏比值与原始CSV。

70条操作即可复现 5000.5倍退化

确定性输入 targeted_cold_large_hot_medium.txt，不含随机选择。K=1、P=4、M=64MiB、W=10000、R=1、penalty=1、B=10000。

Build末尾分区	p0	p1	p2	p3
最终大小	32MiB	16MiB	16MiB	1字节
随后访问p2	不再访问	不再访问	连续4次Probe	不再访问

先顺序构建p0、p1、p2，内存恰好64MiB；再向p3插入1字节触发溢出。该INSERT的next_op是SWITCH，因此调度器看不到Switch后将访问谁。最大元组为1MiB，共65次INSERT、1次SWITCH、4次PROBE。

策略	Build溢出选择	Miss	最终Score
安全朴素 / Prefix	SPILL p0，写32MiB	0	3,355.4432
旧代价模型	SPILL p2，写16MiB	1	16,778,894.7217

旧版通过比较可覆盖缺口的单个分区，选择较便宜的16MiB写出；随后p2的读回耗时为16,777,216微秒。缺1字节空间时再写出p3。总Score/基线 = 5000.500298。

只改Probe目标，优劣立即反转

targeted_hot_large_cold_medium.txt保留相同Build，改为连续访问p0：旧版Score=1,677.7216，基线=33,559,466.1648。不能据一个热点分布决定所有BUILD淘汰策略。

另一个真实随机反例也已保留

探索批次extreme_057：旧版362,690,128.7622，基线43,607.0586，约8317.23倍；Prefix回放恢复为基线同值。其输入、三策略结果和摘要保存在previous_8317x_replay.json。主验证批次旧版最坏为1166.50倍，这些数字来自不同数据组，未混用。

目前的策略选择：收益与风险必须分开

平均收益候选：V2激进双目标

沿用分区写费、读回费和Miss惩罚的代价模型。观测到Miss时增加有界压力，周期衰减；从驻留分区中按单位字节损失排序，比较可覆盖缺口的单分区方案，去掉多余victim。已知下一Probe时，增加目标保护与联合加载分支。

该版本在994个可行输入的963个正分样例上，几何平均代价下降36.57%，但仍有200例退化。它继承BUILD猜错热点的严重风险，不能直接作为“正确且普遍高分”的结论。

稳健底座：PrefixScheduler

BUILD阶段保持安全朴素基线的最大分区淘汰顺序，禁止淘汰正在INSERT的当前分区。Probe Miss仅使用已知的当前A与next_op目标B，比较一条朴素两步轨迹与若干联合驻留轨迹，按真实写、读和Miss代价计算。

接受联合计划的条件	目的
B已处于Probe，A+B≤M	保证联合加载合法
联合后的驻留集合包含模拟基线处理B后的集合	限制额外丢弃其他分区的风险
已知两步真实代价严格更小	不依赖未知的后续热点预测
最后一个动作必须LOAD A	遵守Miss回调格式
周期预取仅使用空闲空间，不新增SPILL	避免提前淘汰导致的付费与信息损失

本轮主集、4个确定性回归、3个最大规模输入中，Prefix均未观察到相对朴素基线退化；主集平均收益仅约0.15%。这是实测结果，不是对任意未来输入不退化的定理。

实现正确性与接口可观察性边界

检查项	执行的约束
输入合法性	读取恰好N条；检查K/P/B/带宽/惩罚、key、元组大小、算子编号与各算子Build/Probe计数
阶段合法性	INSERT必须在Switch前，Probe在后；禁止BUILD阶段LOAD
动作合法性	检查动作索引、溢出回调仅SPILL、Miss最后LOAD当前目标
内存合法性	动作后检查0≤memory_used≤M；中小输入逐动作和逐操作核对所有驻留分区的大小之和
I/O与Score	分别记录写字节、读字节，再独立核算Score；读取过的分区再次SPILL仍完整收费
未来信息	头文件只接收SystemState和当前目标；只能看到一个next_op，不读取驱动的未来操作数组

两个必须处理的题面问题

第一，参考引擎在INSERT回调前进入“驻留”分支，回调后无条件增加驻留字节。若调度器恰好SPILL当前INSERT分区，就会造成账本与真实驻留状态不一致。本次朴素比较基线和所有候选均排除当前目标。原样朴素框架在grow_current_at_full边界输入上已实测触发该问题。

第二，分区必须整块LOAD；如果未来Probe需要的分区大小>M，没有合法整块加载动作。本次单列一个此类输入，8种策略均按预期失败。其余可行输入最终每个分区均≤M；不通过删除失败样例来伪装全覆盖。

Hit没有单独回调，Miss压力不是完整访问频率。未把GDS或ARC依赖逐次访问的更新步骤伪装为已实现。多LOAD动作符合公开参考引擎和“最后一条LOAD当前目标”的文字规则；尚未取得官方SDK作额外兼容认证。

独立交叉检查与小规模精确下界

C++头文件与旧Python模型交叉核对

从小输入与题面范围验证输入中取15例，对朴素、旧版无预取、旧版有预取共45次比较。Hit、Miss和Score全部一致。惰性衰减但关闭新功能的版本，在主验证批次1301个成功输入上与旧版的动作摘要、Hit/Miss和读写字节一致，零差异。

精确下界的计算方式

对最多6个分区枚举所有驻留位掩码。INSERT溢出只允许释放非当前分区；Probe Miss枚举最终包含目标的合法驻留集合；周期点枚举合法SPILL/LOAD集合；逐步做动态规划，保留每个状态的最低累计代价。它允许知道完整未来，是离线最优下界，不能作为在线提交算法。

算法	307个精确小输入中等于离线最优的数量
安全朴素基线	156
旧版 / 周期预取	255
V2 / 激进双目标	256
Prefix / 保守已知前缀	157

所有被测策略的Score均未低于相应离线最优值；没有把“高于下界”解释为已证明算法最优。243例穷举的是固定分区布局下全部3^5个访问序列，不是全部可能题面输入。

变换检查

20组输入分别替换为同余key，8种策略共160次检查中，Score、Hit/Miss、读写字节和动作摘要完全一致。ID旋转的20组也完成合法性回放；固定ID平局规则可能改变选择，因此没有错误地要求旋转后Score必然相同。

复现与完整性

输入和策略均保留摘要；每次SPILL/LOAD的轨迹可由runner再输出。4个确定性输入已保存朴素、旧版、激进版和Prefix的动作CSV。完整逐例结果见scores.csv，原始高精度输出见scores.jsonl。

500万条操作：
实际跑完，而非外推

输入场景	策略	相对代价	回放秒	进程秒
全驻留 / B=1	安全朴素基线	Score=0	1.68	3.61
全驻留 / B=1	旧版 / 周期预取	Score=0	1.81	3.74
全驻留 / B=1	V2 / 空间搭车预取	Score=0	1.64	3.50
全驻留 / B=1	Prefix / 保守已知前缀	Score=0	1.64	3.53
扫描、热点与迁移	安全朴素基线	1.0000	5.99	7.83
扫描、热点与迁移	旧版 / 周期预取	0.6777	20.92	22.78
扫描、热点与迁移	V2 / 空间搭车预取	0.6777	21.66	23.49
扫描、热点与迁移	Prefix / 保守已知前缀	0.9891	18.60	20.45
Build增长 / M=4GiB	安全朴素基线	1.0000	0.06	2.77
Build增长 / M=4GiB	旧版 / 周期预取	0.8056	0.06	2.78
Build增长 / M=4GiB	V2 / 空间搭车预取	0.8056	0.06	2.75
Build增长 / M=4GiB	Prefix / 保守已知前缀	0.9788	0.06	2.86

每例N=5,000,000、K=8、P=64，输入均包含元数据、完整Build、Switch和Probe。混合负载中，Prefix相对基线减少85,492次Miss，读写字节保持相同；旧版取得更大的Score下降，但CPU回放明显慢于朴素基线。

所测进程峰值工作集最高约118.4MiB。这里的哈希数据按字节数模拟记账，没有真实分配4GiB哈希表；该数值仅反映输入数组、调度元数据和回放程序，不能用作真实数据库Hash Join的内存成绩。

回放秒不含输入解析；进程秒包含解析与运行。全部串行测量，但只有本机单次结果，不是题面机器保证。大输入每4096个有效操作完整核对驻留字节账本，每个动作和操作仍检查容量上限；中小输入逐动作完整核对。

交付、复现与当前能支持的结论

当前主验证批次

run_20261018_183734_1b7713

文件 / 目录	用途
cases/ 与三份manifest	所有输入；scale_manifest引用已保存的大输入，SHA-256已核对
results/summary.json、scores.csv	主集汇总、全部逐例结果、输赢和最坏反例
results/final_audit.json	1309个输入、10460次回放、源文件冻结和分数重算核验
results/targeted_results.json	4个确定性回归及动作CSV
results/scale_results.json	3个500万操作输入的实际结果与资源
PrefixScheduler.hpp / CostAwareSchedulerV2.hpp	保守底座与新功能消融版本；原头文件保留
stress20/run_all.cmd	编译后生成新种子数据并运行全部检查；需要本机已有MSVC与Python

重跑现有数据

在design/stress20中执行：

```
python retest.py run
python retest.py summarize
python targeted_checks.py
python scale_tests.py
python final_audit.py
```

要生成全新数据：python retest.py all --fresh。run_all.cmd也会新建批次，不覆盖旧批次。

与文献的关系

Cao与Irani，USITS 1997：GreedyDual-Size：借鉴代价/大小和老化思想，原算法需要每次访问更新。本题可变BUILD大小和写费、受限观察接口不满足直接套用其保证的条件。

Lykouris与Vassilvitskii，ICML 2018：Competitive Caching with Machine Learned Advice：预测建议需要鲁棒后备。本文Prefix是本题已知前缀的具体计划比较，没有继承该论文的竞争比定理。

实际结论：旧策略可作平均收益候选，不能称普遍稳健；保守版解决了已见BUILD灾难性反例，但本轮收益较小。后续高收益改进必须同时保留这些反例与新的独立验证，不接受仅提高平均值而忽略尾部风险的结论。未获取官方隐藏数据，未声称官方高分或全局最优。